IVS Projekt- Kalkulačka

Generated by Doxygen 1.9.8

1 File Index 1
1.1 File List . . . 1

2 File Documentation 3
2.1 gui.c File Reference . . . 3
2.1.1 Detailed Description . . . 4
2.1.2 Function Documentation . . . 4
2.1.2.1 main() . . . 4
2.1.2.2 on_backspace_clicked() . . . 4
2.1.2.3 on_button_clicked() . . . 4
2.1.2.4 on_clear_clicked() . . . 5
2.1.2.5 on_key_press() . . . 5
2.1.2.6 run_calculator() . . . 5
2.1.3 Variable Documentation . . . 5
2.1.3.1 err_msg . . . 5
2.2 gui.h File Reference . . . 6
2.2.1 Detailed Description . . . 7
2.2.2 Function Documentation . . . 7
2.2.2.1 on_backspace_clicked() . . . 7
2.2.2.2 on_button_clicked() . . . 7
2.2.2.3 on_clear_clicked() . . . 8
2.2.2.4 on_key_press() . . . 8
2.2.2.5 run_calculator() . . . 8
2.3 gui.h . . . 9
2.4 math.c File Reference . . . 9
2.4.1 Detailed Description . . . 11
2.4.2 Function Documentation . . . 11
2.4.2.1 add() . . . 11
2.4.2.2 divide() . . . 11
2.4.2.3 factorial() . . . 12
2.4.2.4 logarithm() . . . 12
2.4.2.5 mul() . . . 12
2.4.2.6 n_root() . . . 13
2.4.2.7 power() . . . 13
2.4.2.8 sub() . . . 14
2.5 math.c . . . 14
2.6 math.h File Reference . . . 17
2.6.1 Detailed Description . . . 19
2.6.2 Function Documentation . . . 19
2.6.2.1 add() . . . 19
2.6.2.2 divide() . . . 19
2.6.2.3 factorial() . . . 20

2.6.2.4 logarithm() . . . 20
2.6.2.5 mul() . . . 20
2.6.2.6 n_root() . . . 21
2.6.2.7 power() . . . 21
2.6.2.8 sub() . . . 22
2.7 math.h . . . 22
2.8 parser.c File Reference . . . 22
2.8.1 Detailed Description . . . 24
2.8.2 Function Documentation . . . 24
2.8.2.1 parse_add() . . . 24
2.8.2.2 parse_div() . . . 24
2.8.2.3 parse_equation() . . . 25
2.8.2.4 parse_factorial() . . . 25
2.8.2.5 parse_log() . . . 26
2.8.2.6 parse_mul() . . . 26
2.8.2.7 parse_number() . . . 27
2.8.2.8 parse_power() . . . 27
2.8.2.9 parse_root() . . . 27
2.8.2.10 parse_sub() . . . 28
2.9 parser.c . . . 28
2.10 parser.h File Reference . . . 32
2.10.1 Detailed Description . . . 34
2.10.2 Function Documentation . . . 34
2.10.2.1 parse_add() . . . 34
2.10.2.2 parse_div() . . . 35
2.10.2.3 parse_equation() . . . 35
2.10.2.4 parse_factorial() . . . 35
2.10.2.5 parse_log() . . . 36
2.10.2.6 parse_mul() . . . 36
2.10.2.7 parse_number() . . . 37
2.10.2.8 parse_power() . . . 37
2.10.2.9 parse_root() . . . 38
2.10.2.10 parse_sub() . . . 38
2.11 parser.h . . . 38
2.12 test_math.c File Reference . . . 39
2.12.1 Function Documentation . . . 40
2.12.1.1 absolute() . . . 40
2.12.1.2 add() . . . 40
2.12.1.3 divide() . . . 40
2.12.1.4 factorial() . . . 40
2.12.1.5 is_equal() . . . 40
2.12.1.6 logarithm() . . . 40

2.12.1.7 main() 40
2.12.1.8 mul() 41
2.12.1.9 n_root() 41
2.12.1.10 power() 41
2.12.1.11 sub() 41

Index **43**

Chapter 1

File Index

1.1 File List

Here is a list of all files with brief descriptions:

gui.c
Graphical user interface of the calculator app 3
gui.h
Header file for the graphical user interface of the calculator app 6
math.c
Library for calculting math functions 9
math.h
Library for calculating mathematical functions 17
parser.c
Parser of the calculator app 22
parser.h
Parser of the calculator app 32
test_math.c 39

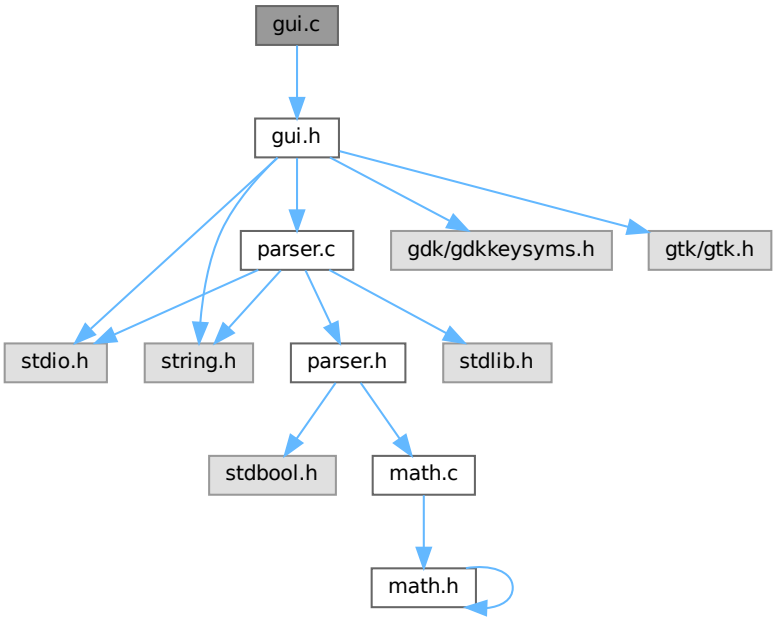
Chapter 2

File Documentation

2.1 gui.c File Reference

graphical user interface of the calculator app

`#include "gui.h"`
Include dependency graph for gui.c:



Functions

- void on_button_clicked (GtkWidget ∗button, GtkWidget ∗display)

  Handles button clicks (numbers and operators)
- void on_clear_clicked (GtkWidget ∗button, GtkWidget ∗display)

Clears the display.

- void on_backspace_clicked (GtkWidget ∗button, GtkWidget ∗display)

  Deletes the last character from the display.
- gboolean on_key_press (GtkWidget ∗window, GdkEventKey ∗event, GtkWidget ∗display)

  Deletes the last character from the display.
- int run_calculator (int argc, char ∗argv[ ])

  Initializes and runs the calculator GUI.
- int main (int argc, char ∗argv[ ])

Variables

- const char ∗ err_msg [ ]

2.1.1 Detailed Description

graphical user interface of the calculator app

Author

Daniel Kratky

2.1.2 Function Documentation

2.1.2.1 main()

```
int main (
            int argc,
            char * argv[] )
```

Program entry point

2.1.2.2 on_backspace_clicked()

```
void on_backspace_clicked (
            GtkWidget * button,
            GtkWidget * display )
```

Deletes the last character from the display.

Deletes last character on the display

2.1.2.3 on_button_clicked()

```
void on_button_clicked (
            GtkWidget * button,
            GtkWidget * display )
```

Handles button clicks (numbers and operators)

Handles button clicks (numbers and operators)

2.1.2.4 on_clear_clicked()

```
void on_clear_clicked (
            GtkWidget * button,
            GtkWidget * display )
```

Clears the display.

Clears the display

2.1.2.5 on_key_press()

```
gboolean on_key_press (
            GtkWidget * window,
            GdkEventKey * event,
            GtkWidget * display )
```

Deletes the last character from the display.

Removes the last character in the entry field if it exists.

Parameters

window	Pointer to the window widget
event	Pointer to the event that triggered the signal
display	Pointer to the display entry all three parameters are necessary to folow the gtk conventions and work with other functions they can accept null pointers if unused

2.1.2.6 run_calculator()

```
int run_calculator (
            int argc,
            char * argv[] )
```

Initializes and runs the calculator GUI.

Initializes and runs the calculator GUI

2.1.3 Variable Documentation

2.1.3.1 err_msg

```
const char* err_msg[]
```

Initial value:

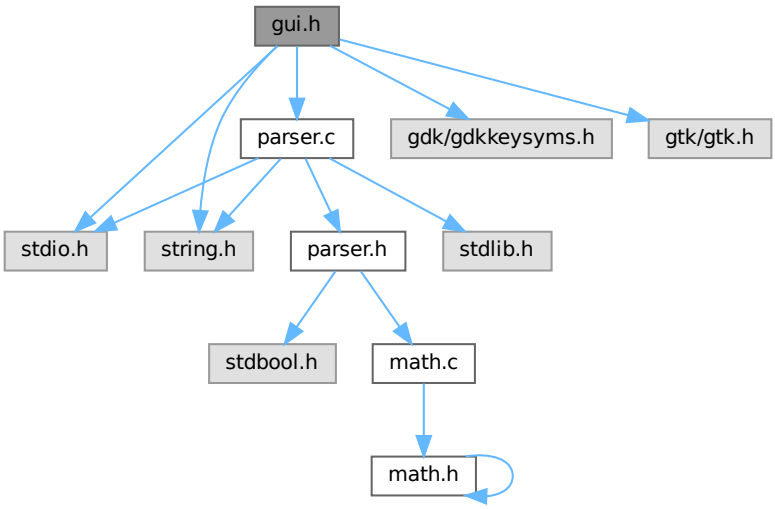
```
= {
    "Error: Multimle decimal points",
    "Error: Unrecognized character",
    "Error: No input for function",
    "Error: Number out of bounds",
    "Error: Division by 0",
    "Error: Decimal number in factorial",
    "Error: Negative number in factorial",
    "Error: Negative number or 0 in root",
    "Error: Root does not exist",
    "Error: Negative number or 0 in log",
    "Error: Decimal number in exponent",
    "Error: Decimal number in degree"
}
```

2.2 gui.h File Reference

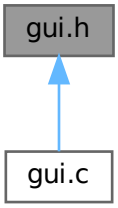
Header file for the graphical user interface of the calculator app.

```
#include <stdio.h>
#include <string.h>
#include <gdk/gdkkeysyms.h>
#include <gtk/gtk.h>
#include "parser.c"
```

Include dependency graph for gui.h:



This graph shows which files directly or indirectly include this file:



Functions

- void on_button_clicked (GtkWidget *button, GtkWidget *display)

  Handles button clicks (numbers and operators)
- void on_clear_clicked (GtkWidget *button, GtkWidget *display)

Clears the display.

- void on_backspace_clicked (GtkWidget ∗button, GtkWidget ∗display)

  Deletes the last character from the display.
- gboolean on_key_press (GtkWidget ∗window, GdkEventKey ∗event, GtkWidget ∗display)

  Deletes the last character from the display.
- int run_calculator (int argc, char ∗argv[ ])

  Initializes and runs the calculator GUI.

2.2.1 Detailed Description

Header file for the graphical user interface of the calculator app.

Author

Daniel Kratky

This file declares the global variables and function prototypes used in the calculator program. The calculator provides a basic graphical user interface (GUI) with buttons and a display field

2.2.2 Function Documentation

2.2.2.1 on_backspace_clicked()

```
void on_backspace_clicked (
            GtkWidget * button,
            GtkWidget * display )
```

Deletes the last character from the display.

Removes the last character in the entry field if it exists.

Parameters

button	Pointer to the button widget
display	Pointer to the display entry both parameters are necessary to folow the gtk conventions and work with other functions they can accept null pointers if unused

Deletes last character on the display

2.2.2.2 on_button_clicked()

```
void on_button_clicked (
            GtkWidget * button,
            GtkWidget * display )
```

Handles button clicks (numbers and operators)

Appends the pressed button's label to the display.

Parameters

button	Pointer to the button widget
display	Pointer to the display entry both parameters are necessary to folow the gtk conventions and work with other functions they can accept null pointers if unused

Handles button clicks (numbers and operators)

2.2.2.3 on_clear_clicked()

```
void on_clear_clicked (
            GtkWidget * button,
            GtkWidget * display )
```

Clears the display.

Sets the display text to an empty string.

Parameters

button	Pointer to the button widget
display	Pointer to the display entry both parameters are necessary to folow the gtk conventions and work with other functions they can accept null pointers if unused

Clears the display

2.2.2.4 on_key_press()

```
gboolean on_key_press (
            GtkWidget * window,
            GdkEventKey * event,
            GtkWidget * display )
```

Deletes the last character from the display.

Removes the last character in the entry field if it exists.

Parameters

window	Pointer to the window widget
event	Pointer to the event that triggered the signal
display	Pointer to the display entry all three parameters are necessary to folow the gtk conventions and work with other functions they can accept null pointers if unused

2.2.2.5 run_calculator()

```
int run_calculator (
            int argc,
            char * argv[] )
```

Initializes and runs the calculator GUI.

Creates the window, sets up all widgets, and starts the GTK main event loop.

Parameters

argc	Argument count from main()
argv	Argument vector from main()

Returns

int Exit status

Initializes and runs the calculator GUI

2.3 gui.h

Go to the documentation of this file.

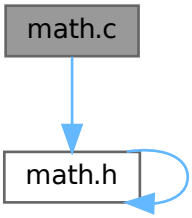
```
//======= Copyright (c) 2026, FIT VUT Brno, GNU GPL V3.0 ============//
//
// Purpose:     calculator GUI
//
// $NoKeywords: $ivs_project_2 $gui.h
// $Authors:    Daniel Kratky <xkratkd00@stud.fit.vut.cz>
// $Date:       $2026-04-23
//===========================================================================//
#ifndef GUI_H
#define GUI_H

#include <stdio.h>
#include <string.h>
#include <gdk/gdkkeysyms.h>
#include <gtk/gtk.h>
#include "parser.c"

void on_button_clicked(GtkWidget *button, GtkWidget *display);

void on_clear_clicked(GtkWidget *button, GtkWidget *display);

void on_backspace_clicked(GtkWidget *button, GtkWidget *display);

gboolean on_key_press(GtkWidget *window, GdkEventKey *event, GtkWidget *display);

int run_calculator(int argc, char *argv[]);

#endif // GUI_H
```

2.4 math.c File Reference

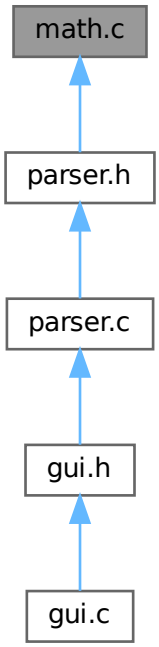
Library for calculting math functions.

```
#include "math.h"
```

Include dependency graph for math.c:



This graph shows which files directly or indirectly include this file:



Functions

- double add (double a, double b, int *error)

  Adds two numbers.
- double sub (double a, double b, int *error)

  Subtracts two numbers.
- double mul (double a, double b, int *error)

Multiplies two numbers.

- double divide (double a, double b, int ∗error)

  Divides two numbers.
- double factorial (double n, int ∗error)

  Computes factorial of a number.
- double power (double base, double exponent, int ∗error)

  Computes power function.
- double n_root (double degree, double radicand, int ∗error)

  Computes n-th root of a number.
- double logarithm (double x, int ∗error)

  Computes base-10 logarithm.

2.4.1 Detailed Description

Library for calculting math functions.

Author

Daniel Kratky, Radek Zak

2.4.2 Function Documentation

2.4.2.1 add()

```
double add (
            double a,
            double b,
            int * error )
```

Adds two numbers.

Parameters

a	First addend
b	Second addend
error	Pointer to error code (0 if no error occurred)

Returns

Sum of a and b

2.4.2.2 divide()

```
double divide (
            double a,
            double b,
            int * error )
```

Divides two numbers.

Parameters

a	Numerator
b	Denominator
error	Pointer to error code (0 if no error occurred)

Returns

Result of division (a / b)

2.4.2.3 factorial()

```
double factorial (
            double n,
            int * error )
```

Computes factorial of a number.

Parameters

n	Non-negative integer
error	Pointer to error code (0 if no error occurred)

Returns

n!

2.4.2.4 logarithm()

```
double logarithm (
            double x,
            int * error )
```

Computes base-10 logarithm.

Parameters

x	Input value (must be $>$ 0)
error	Pointer to error code (0 if no error occurred)

Returns

log10(x)

2.4.2.5 mul()

```
double mul (
            double a,
```

```
            double b,
            int * error )
```

Multiplies two numbers.

Parameters

a	First factor
b	Second factor
error	Pointer to error code (0 if no error occurred)

Returns

Product of a and b

2.4.2.6 n_root()

```
double n_root (
            double degree,
            double radicand,
            int * error )
```

Computes n-th root of a number.

Parameters

degree	Root degree (e.g., 2 for square root)
radicand	Value under the root
error	Pointer to error code (0 if no error occurred)

Returns

n-th root of radicand

2.4.2.7 power()

```
double power (
            double base,
            double exponent,
            int * error )
```

Computes power function.

Parameters

base	Base value
exponent	Integer exponent
error	Pointer to error code (0 if no error occurred)

Returns

base raised to the power exponent

2.4.2.8 sub()

```
double sub (
            double a,
            double b,
            int * error )
```

Subtracts two numbers.

Parameters

a	Minuend
b	Subtrahend
error	Pointer to error code (0 if no error occurred)

Returns

Difference (a - b)

2.5 math.c

Go to the documentation of this file.

```
00001 //======= Copyright (c) 2026, FIT VUT Brno, GNU GPL V3.0 ============//
00002 //
00003 // Purpose:     math library
00004 //
00005 // $NoKeywords: $ivs_project_2 $math.c
00006 // $Authors:    Radek Zak <xzakrad00@stud.fit.vut.cz>, Daniel Kratky <xkratkd00@stud.fit.vut.cz>
00007 // $Date:       $2026-04-24
00008 //===========================================================================//
00015 #include "math.h"
00016 #include <math.h>
00017
00018 double add(double a, double b, int *error) {
00019     double result = a + b;
00020
00021     if(error){
00022         // overflow check (positive and negative)
00023         if (isinf(result)) {
00024             *error = 4; // overflow error
00025         } else {
00026             *error = 0;
00027         }
00028     }
00029
00030     return result;
00031 }
00032 double sub(double a, double b, int *error) {
00033     double result = a - b;
00034
00035     if(error){
00036         // overflow check (positive and negative)
00037         if (isinf(result)) {
00038             *error = 4; // overflow error
00039         } else {
00040             *error = 0;
00041         }
00042     }
00043
00044     return result;
00045 }
00046 double mul(double a, double b, int *error) {
```

```c
    double result = a * b;

    if(error){
        // overflow check (positive and negative)
        if (isinf(result)) {
            *error = 4; // overflow error
        } else {
            *error = 0;
        }
    }

    return result;
}
double divide(double a, double b, int *error) {
    if (b == 0.0) {
        if (error) {
            *error = 5; // error: div by zero
        }
        return 0.0;
    }
    else {
        if (error) {
            *error = 0;
        }
        double result = a / b;
        return result;
    }
}
double factorial(double n, int *error) {
    if (n != (int)n){
        *error = 6; // Decimal number in factorial
        return 0.0;
    }
    if (n < 0){
        if (error){
            *error = 7; // Negative number in factorial
        }
        return 0;
    }
    if (n == 0 || n == 1) {
        if (error){
            *error = 0;
        }
        return 1;
    }
    //overflow check (21! doesn't fit double)
    if (n > 20) {
        if (error){
            *error = 4; // overflow error
        }
        return 0;
    }
    double result = 1;
    for (int i = 2; i <= n; i++) {
        result *= i;
    }

    if (error) {
        *error = 0;
    }
    return result;
}
double power(double base, double exponent, int *error) {
    if (exponent != (int)exponent){
        *error = 11; // Decimal number in exponent
        return 0.0;
    }
    if (exponent == 0) {
        if (error) {
            *error = 0;
        }
        return 1.0;
    }
    // error div by zero
    if (base == 0.0 && exponent < 0) {
        if (error){
            *error = 5; // Division by 0
        }
        return 0.0;
    }

    double result = 1.0;

    if (exponent > 0) {
        for (int i = 0; i < exponent; i++) {
            result *= base;
        }
```

```

    }
    else {
        for (int i = 0; i < -exponent; i++) {
            result *= base;
        }

        result = 1.0 / result;
    }
    if (error) {
        if (isinf(result)) {
            *error = 4; // overflow error
        }
        else {
            *error = 0;
        }
    }
    return result;
}
double n_root(double degree, double radicand, int *error) {
    if (degree != (int)degree){
        *error = 12; // Decimal number in degree
        return 0.0;
    }
    if (degree <= 0) {
        if (error) {
            *error = 8; // negative number or 0 in root
        }
        return 0.0;
    }

    if (radicand < 0 && (int)degree % 2 == 0) {
        if (error) {
            *error = 9; // root does not exist
        }
        return 0.0;
    }

    if (radicand == 0.0) {
        if (error) {
            *error = 0;}
        return 0.0;
    }

    double x = radicand;
    double precision = 0.00001;

    for (int i = 0; i < 50; i++) {
        double prev = x;

        double previous_power = 1.0;
        for (int j = 0; j < degree - 1; j++) {
            previous_power *= x;
        }

        if (previous_power == 0.0) {
            break;
        }

        x = ((degree - 1) * x + radicand / previous_power) / degree;

        double diff = x - prev;
        if (diff < 0) {
            diff = -diff;
        }

        if (diff < precision) {
            break;
        }
    }

    if (error) {
        *error = 0;}

    return x;
}
double logarithm(double x, int *error) {
    if (x <= 0) {
        if (error) {
            *error = 10; // Negative number or 0 in log
        }
        return 0.0;
    }

    if (error) {
        *error = 0;
    }
```

```

    double result = 0.0;
    double fraction = (x - 1.0) / (x + 1.0);
    double fraction_power = fraction * fraction;
    double numerator = fraction;

    for (int i = 1; i < 40; i += 2) {
        result += numerator / i;
        numerator *= fraction_power;
    }
    double ln_x = 2.0 * result;

    // Convert ln(x) to log10(x)
    const double LN10 = 2.302585092994046; // ln(10)
    return ln_x / LN10;
}
```

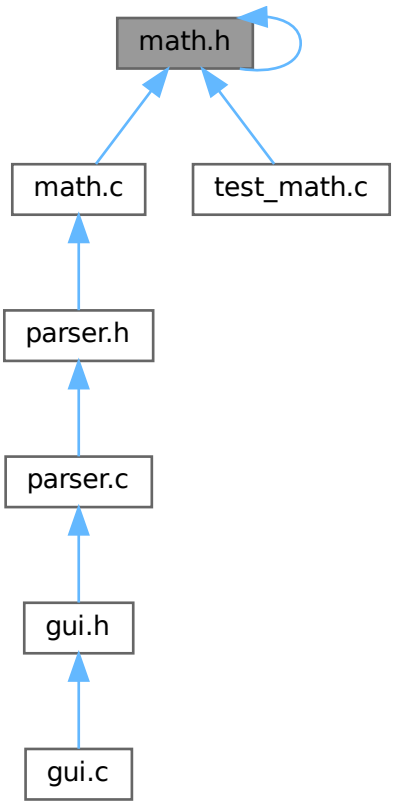
2.6 math.h File Reference

Library for calculating mathematical functions.

`#include <math.h>`

Include dependency graph for math.h:

This graph shows which files directly or indirectly include this file:



Functions

- double add (double a, double b, int ∗error)

  Adds two numbers.
- double sub (double a, double b, int ∗error)

  Subtracts two numbers.
- double mul (double a, double b, int ∗error)

  Multiplies two numbers.
- double divide (double a, double b, int ∗error)

  Divides two numbers.
- double factorial (double n, int ∗error)

  Computes factorial of a number.
- double power (double base, double exponent, int ∗error)

  Computes power function.
- double n_root (double degree, double radicand, int ∗error)

  Computes n-th root of a number.
- double logarithm (double x, int ∗error)

  Computes base-10 logarithm.

2.6.1 Detailed Description

Library for calculating mathematical functions.

Author

Daniel Kratky

This file declares function prototypes for basic arithmetic and advanced mathematical operations used in the calculator.

2.6.2 Function Documentation

2.6.2.1 add()

```
double add (
            double a,
            double b,
            int * error )
```

Adds two numbers.

Parameters

a	First addend
b	Second addend
error	Pointer to error code (0 if no error occurred)

Returns

Sum of a and b

2.6.2.2 divide()

```
double divide (
            double a,
            double b,
            int * error )
```

Divides two numbers.

Parameters

a	Numerator
b	Denominator
error	Pointer to error code (0 if no error occurred)

Returns

Result of division (a / b)

2.6.2.3 factorial()

```
double factorial (
            double n,
            int * error )
```

Computes factorial of a number.

Parameters

n	Non-negative integer
error	Pointer to error code (0 if no error occurred)

Returns

n!

2.6.2.4 logarithm()

```
double logarithm (
            double x,
            int * error )
```

Computes base-10 logarithm.

Parameters

x	Input value (must be $>$ 0)
error	Pointer to error code (0 if no error occurred)

Returns

log10(x)

2.6.2.5 mul()

```
double mul (
            double a,
            double b,
            int * error )
```

Multiplies two numbers.

Parameters

a	First factor
b	Second factor
error	Pointer to error code (0 if no error occurred)

Returns

Product of a and b

2.6.2.6 n_root()

```
double n_root (
            double degree,
            double radicand,
            int * error )
```

Computes n-th root of a number.

Parameters

degree	Root degree (e.g., 2 for square root)
radicand	Value under the root
error	Pointer to error code (0 if no error occurred)

Returns

n-th root of radicand

2.6.2.7 power()

```
double power (
            double base,
            double exponent,
            int * error )
```

Computes power function.

Parameters

base	Base value
exponent	Integer exponent
error	Pointer to error code (0 if no error occurred)

Returns

base raised to the power exponent

2.6.2.8 sub()

```
double sub (
            double a,
            double b,
            int * error )
```

Subtracts two numbers.

Parameters

a	Minuend
b	Subtrahend
error	Pointer to error code (0 if no error occurred)

Returns

Difference (a - b)

2.7 math.h

Go to the documentation of this file.

```
00001 //======= Copyright (c) 2026, FIT VUT Brno, GNU GPL V3.0 ============//
00002 //
00003 // Purpose:     math library
00004 //
00005 // $NoKeywords: $ivs_project_2 $math.h
00006 // $Authors:    Daniel Kratky <xkratkd00@stud.fit.vut.cz>
00007 // $Date:       $2026-04-24
00008 //===========================================================================//
00009 #ifndef MATH_H
00010 #define MATH_H
00021 #include <math.h>
00022
00032 double add(double a, double b, int *error);
00033
00043 double sub(double a, double b, int *error);
00044
00054 double mul(double a, double b, int *error);
00055
00065 double divide(double a, double b, int *error);
00066
00075 double factorial(double n, int *error);
00076
00086 double power(double base, double exponent, int *error);
00087
00097 double n_root(double degree, double radicand, int *error);
00098
00107 double logarithm(double x, int *error);
00108
00109 #endif // MATH_H
```

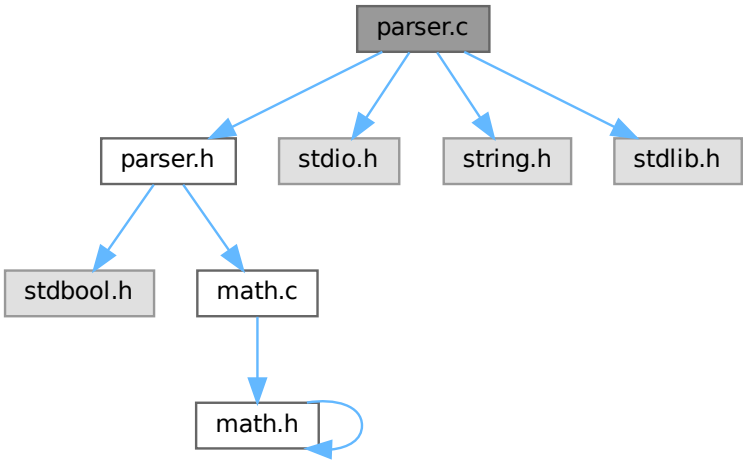
2.8 parser.c File Reference

parser of the calculator app

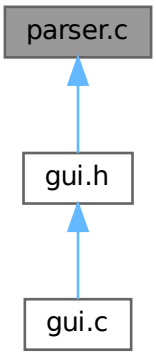
```
#include "parser.h"
#include <stdio.h>
#include <string.h>
```

```
#include <stdlib.h>
```

Include dependency graph for parser.c:



This graph shows which files directly or indirectly include this file:



Functions

- double parse_factorial (const char ∗equation, unsigned int lenght, unsigned int ∗error, bool ∗parsed)

  searches the string equation for factorials and evaluates them
- double parse_log (const char ∗equation, unsigned int lenght, unsigned int ∗error, bool ∗parsed)

  searches the string equation for logarithms and evaluates them
- double parse_power (const char ∗equation, unsigned int lenght, unsigned int ∗error, bool ∗parsed)

  searches the string equation for powers and evaluates them
- double parse_root (const char ∗equation, unsigned int lenght, unsigned int ∗error, bool ∗parsed)

searches the string equation for roots and evaluates them

- double parse_mul (const char ∗equation, unsigned int lenght, unsigned int ∗error, bool ∗parsed)

  searches the string equation for multiplications and evaluates them
- double parse_div (const char ∗equation, unsigned int lenght, unsigned int ∗error, bool ∗parsed)

  searches the string equation for divisions and evaluates them
- double parse_add (const char ∗equation, unsigned int lenght, unsigned int ∗error, bool ∗parsed)

  searches the string equation for additions and evaluates them
- double parse_sub (const char ∗equation, unsigned int lenght, unsigned int ∗error, bool ∗parsed)

  searches the string equation for subtractions and evaluates them
- double parse_equation (const char ∗equation, unsigned int lenght, unsigned int depth, unsigned int ∗error)

  parses a string into a mathematical equation and returns its value
- double parse_number (const char ∗number, unsigned int lenght, unsigned int ∗error)

  parses a string into decimal number

2.8.1 Detailed Description

parser of the calculator app

Author

Daniel Kratky

2.8.2 Function Documentation

2.8.2.1 parse_add()

```
double parse_add (
            const char * equation,
            unsigned int lenght,
            unsigned int * error,
            bool * parsed )
```

searches the string equation for additions and evaluates them

Parameters

equation	Pointer to the string containing the equation
lenght	Lenght of the string containing the equation
error	Pointer to error code (0 if no error occurred)
parsed	Pointer bool set to true if the function parsed anythng, false otherwise

Returns

value of the equation

2.8.2.2 parse_div()

```
double parse_div (
            const char * equation,
```

```
            unsigned int lenght,
            unsigned int * error,
            bool * parsed )
```

searches the string equation for divisions and evaluates them

Parameters

equation	Pointer to the string containing the equation
lenght	Lenght of the string containing the equation
error	Pointer to error code (0 if no error occurred)
parsed	Pointer bool set to true if the function parsed anythng, false otherwise

Returns

value of the equation

2.8.2.3 parse_equation()

```
double parse_equation (
            const char * equation,
            unsigned int lenght,
            unsigned int depth,
            unsigned int * error )
```

parses a string into a mathematical equation and returns its value

Parameters

equation	Pointer to the string containing the equation
lenght	Lenght of the string containing the equation
depth	Second addend
error	Pointer to error code (0 if no error occurred)

Returns

value of the equation

2.8.2.4 parse_factorial()

```
double parse_factorial (
            const char * equation,
            unsigned int lenght,
            unsigned int * error,
            bool * parsed )
```

searches the string equation for factorials and evaluates them

Parameters

equation	Pointer to the string containing the equation
lenght	Lenght of the string containing the equation
error	Pointer to error code (0 if no error occurred)
parsed	Pointer bool set to true if the function parsed anythng, false otherwise

Returns

value of the equation

2.8.2.5 parse_log()

```
double parse_log (
            const char * equation,
            unsigned int lenght,
            unsigned int * error,
            bool * parsed )
```

searches the string equation for logarithms and evaluates them

Parameters

equation	Pointer to the string containing the equation
lenght	Lenght of the string containing the equation
error	Pointer to error code (0 if no error occurred)
parsed	Pointer bool set to true if the function parsed anythng, false otherwise

Returns

value of the equation

2.8.2.6 parse_mul()

```
double parse_mul (
            const char * equation,
            unsigned int lenght,
            unsigned int * error,
            bool * parsed )
```

searches the string equation for multiplications and evaluates them

Parameters

equation	Pointer to the string containing the equation
lenght	Lenght of the string containing the equation
error	Pointer to error code (0 if no error occurred)
parsed	Pointer bool set to true if the function parsed anythng, false otherwise

Returns

value of the equation

2.8.2.7 parse_number()

```
double parse_number (
            const char * number,
            unsigned int lenght,
            unsigned int * error )
```

parses a string into decimal number

Parameters

number	Pointer to the string containing the number
lenght	Lenght of the string containing the equation
depth	Second addend
error	Pointer to error code (0 if no error occurred)

Returns

decimal value of the number

2.8.2.8 parse_power()

```
double parse_power (
            const char * equation,
            unsigned int lenght,
            unsigned int * error,
            bool * parsed )
```

searches the string equation for powers and evaluates them

Parameters

equation	Pointer to the string containing the equation
lenght	Lenght of the string containing the equation
error	Pointer to error code (0 if no error occurred)
parsed	Pointer bool set to true if the function parsed anythng, false otherwise

Returns

value of the equation

2.8.2.9 parse_root()

```
double parse_root (
            const char * equation,
```

```
            unsigned int lenght,
            unsigned int * error,
            bool * parsed )
```

searches the string equation for roots and evaluates them

Parameters

equation	Pointer to the string containing the equation
lenght	Lenght of the string containing the equation
error	Pointer to error code (0 if no error occurred)
parsed	Pointer bool set to true if the function parsed anythng, false otherwise

Returns

value of the equation

2.8.2.10 parse_sub()

```
double parse_sub (
            const char * equation,
            unsigned int lenght,
            unsigned int * error,
            bool * parsed )
```

searches the string equation for subtractions and evaluates them

Parameters

equation	Pointer to the string containing the equation
lenght	Lenght of the string containing the equation
error	Pointer to error code (0 if no error occurred)
parsed	Pointer bool set to true if the function parsed anythng, false otherwise

Returns

value of the equation

2.9 parser.c

Go to the documentation of this file.

```
00001 //======= Copyright (c) 2026, FIT VUT Brno, GNU GPL V3.0 ============//
00002 //
00003 // Purpose:     calculator parser
00004 //
00005 // $NoKeywords: $ivs_project_2 $parser.c
00006 // $Authors:    Daniel Kratky <xkratkd00@stud.fit.vut.cz>
00007 // $Date:       $2026-04-28
00008 //===========================================================================//
00015 #include "parser.h"
00016 #include <stdio.h>
00017 #include <string.h>
00018 #include <stdlib.h>
```

```c


double parse_factorial(const char *equation, unsigned int lenght, unsigned int *error, bool *parsed) {
    for (int character = lenght - 1; character >= 0; character--) {
        if (equation[character] == '!') {
            *parsed = true;

            // if there is nothing as input
            if (character == 0) {
                *error = 3;
                return 0.0;
            }

            double num_l = parse_equation(equation, character, 7, error);
            if(*error) {
                return 0.0;
            }
            return factorial(num_l, error);
        }
    }
    *parsed = false;
    return 0.0;
}

double parse_log(const char *equation, unsigned int lenght, unsigned int *error, bool *parsed) {
    for (int character = lenght - 1; character >= 0; character--) {
        if (equation[character] == 'l') {
            *parsed = true;
            const char *equation_r = equation + character + 1;
            unsigned int lenght_r = lenght - character - 1;
            if (lenght_r == 0) {
                *error = 3;
                return 0.0;
            }
            double num_r = parse_equation(equation_r, lenght_r, 6, error);
            if(*error) {
                return 0.0;
            }
            return logarithm(num_r, error);
        }
    }
    *parsed = false;
    return 0.0;
}

double parse_power(const char *equation, unsigned int lenght, unsigned int *error, bool *parsed) {
    for (int character = lenght - 1; character >= 0; character--) {
        if (equation[character] == '^') {
            *parsed = true;

            const char *equation_r = equation + character + 1;
            const char *equation_l = equation;
            unsigned int lenght_r = lenght - character - 1;
            unsigned int lenght_l = character;

            // if there is nothing as input
            if (lenght_r == 0 || lenght_l == 0) {
                *error = 3;
                return 0.0;
            }

            double num_r = parse_equation(equation_r, lenght_r, 6, error);
            if(*error) {
                return 0.0;
            }

            double num_l = parse_equation(equation_l, lenght_l, 5, error);
            if(*error) {
                return 0.0;
            }
            return power(num_l, num_r, error);
        }
    }
    *parsed = false;
    return 0.0;
}

double parse_root(const char *equation, unsigned int lenght, unsigned int *error, bool *parsed) {
    for (int character = lenght - 1; character >= 0; character--) {
        if (equation[character] == 'r') {
            *parsed = true;

            const char *equation_r = equation + character + 1;
            const char *equation_l = equation;
            unsigned int lenght_r = lenght - character - 1;
            unsigned int lenght_l = character;

```

```
            // if there is nothing as input
            if (lenght_r == 0) {
                *error = 3;
                return 0.0;
            }

            double num_r = parse_equation(equation_r, lenght_r, 5, error);
            if(*error) {
                return 0.0;
            }

            double num_l = parse_equation(equation_l, lenght_l, 4, error);
            if(*error) {
                return 0.0;
            }
            return n_root(num_l, num_r, error);
        }
    }
    *parsed = false;
    return 0.0;
}

double parse_mul(const char *equation, unsigned int lenght, unsigned int *error, bool *parsed) {
    for (int character = lenght - 1; character >= 0; character--) {
        if (equation[character] == '*') {
            *parsed = true;

            const char *equation_r = equation + character + 1;
            const char *equation_l = equation;
            unsigned int lenght_r = lenght - character - 1;
            unsigned int lenght_l = character;

            // if there is nothing as input
            if (lenght_r == 0 || lenght_l == 0) {
                *error = 3;
                return 0.0;
            }

            double num_r = parse_equation(equation_r, lenght_r, 4, error);
            if(*error) {
                return 0.0;
            }

            double num_l = parse_equation(equation_l, lenght_l, 3, error);
            if(*error) {
                return 0.0;
            }
            return mul(num_l, num_r, error);
        }
    }
    *parsed = false;
    return 0.0;
}

double parse_div(const char *equation, unsigned int lenght, unsigned int *error, bool *parsed) {
    for (int character = lenght - 1; character >= 0; character--) {
        if (equation[character] == '/') {
            *parsed = true;

            const char *equation_r = equation + character + 1;
            const char *equation_l = equation;
            unsigned int lenght_r = lenght - character - 1;
            unsigned int lenght_l = character;

            // if there is nothing as input
            if (lenght_r == 0 || lenght_l == 0) {
                *error = 3;
                return 0.0;
            }

            double num_r = parse_equation(equation_r, lenght_r, 3, error);
            if(*error) {
                return 0.0;
            }

            double num_l = parse_equation(equation_l, lenght_l, 2, error);
            if(*error) {
                return 0.0;
            }

            return divide(num_l, num_r, error);
        }
    }
    *parsed = false;
    return 0.0;
}
```

```c
double parse_add(const char *equation, unsigned int lenght, unsigned int *error, bool *parsed) {
    for (int character = lenght - 1; character >= 0; character--) {
        if (equation[character] == '+') {
            *parsed = true;

            const char *equation_r = equation + character + 1;
            const char *equation_l = equation;
            unsigned int lenght_r = lenght - character - 1;
            unsigned int lenght_l = character;

            // if there is nothing as input
            if (lenght_r == 0 || lenght_l == 0) {
                *error = 3;
                return 0.0;
            }

            double num_r = parse_equation(equation_r, lenght_r, 2, error);
            if(*error) {
                return 0.0;
            }

            double num_l = parse_equation(equation_l, lenght_l, 1, error);
            if(*error) {
                return 0.0;
            }
            return add(num_l, num_r, error);
        }
    }
    *parsed = false;
    return 0.0;
}

double parse_sub(const char *equation, unsigned int lenght, unsigned int *error, bool *parsed) {
    for (int character = lenght - 1; character >= 0; character--) {
        if (equation[character] == '-') {

            // Skip unary minus
            if (character == 0 ||
                equation[character - 1] == '+' ||
                equation[character - 1] == '-' ||
                equation[character - 1] == '*' ||
                equation[character - 1] == '/' ||
                equation[character - 1] == '^' ||
                equation[character - 1] == 'r' ||
                equation[character - 1] == 'l') {
                continue;
            }
            *parsed = true;

            const char *equation_r = equation + character + 1;
            const char *equation_l = equation;
            unsigned int lenght_r = lenght - character - 1;
            unsigned int lenght_l = character;

            // if there is nothing as input (continue on unary minus, no need to check it)
            if (lenght_r == 0) {
                *error = 3;
                return 0.0;
            }

            double num_r = parse_equation(equation_r, lenght_r, 1, error);
            if(*error) {
                return 0.0;
            }

            double num_l = parse_equation(equation_l, lenght_l, 0, error);
            if(*error) {
                return 0.0;
            }
            return sub(num_l, num_r, error);
        }
    }
    *parsed = false;
    return 0.0;
}

double parse_equation(const char *equation, unsigned int lenght, unsigned int depth, unsigned int *error) {
    bool parsed;
    double value;
    // parses for each operator, ignores the one for which it already checked
    switch(depth) {
        case 0:
            value = parse_sub(equation, lenght, error, &parsed);
            if (parsed) {
                return value;
            }
```

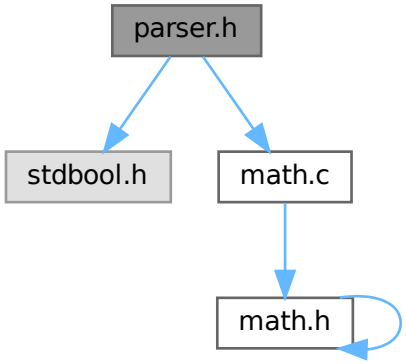
```
        case 1:
            value = parse_add(equation, lenght, error, &parsed);
            if (parsed) {
                return value;
            }
        case 2:
            value = parse_div(equation, lenght, error, &parsed);
            if (parsed) {
                return value;
            }
        case 3:
            value = parse_mul(equation, lenght, error, &parsed);
            if (parsed) {
                return value;
            }
        case 4:
            value = parse_root(equation, lenght, error, &parsed);
            if (parsed) {
                return value;
            }
        case 5:
            value = parse_power(equation, lenght, error, &parsed);
            if (parsed) {
                return value;
            }
        case 6:
            value = parse_log(equation, lenght, error, &parsed);
            if (parsed) {
                return value;
            }
        case 7:
            value = parse_factorial(equation, lenght, error, &parsed);
            if (parsed) {
                return value;
            }
        default:
            return parse_number(equation, lenght, error);
    }
}

double parse_number(const char *number, unsigned int lenght, unsigned int *error) {
    int decimal_points = 0;
    for (unsigned int character = 0; character < lenght; character++) {
        // Allow unary minus if it is first
        if (character == 0 && number[character] == '-') {
            continue;
        }

        // Catch multiple decimals in one number
        if (number[character] == '.') {
            decimal_points++;
        }

        // Catch nonstandard characters
        if (!(number[character] == '.' ||
              number[character] == '0' ||
              number[character] == '1' ||
              number[character] == '2' ||
              number[character] == '3' ||
              number[character] == '4' ||
              number[character] == '5' ||
              number[character] == '6' ||
              number[character] == '7' ||
              number[character] == '8' ||
              number[character] == '9')) {
            *error = 2;
            return 0.0;
        }
    }
    if (decimal_points > 1) {
        *error = 1;
        return 0.0;
    }
    return strtod(number, NULL);
}
```

2.10 parser.h File Reference

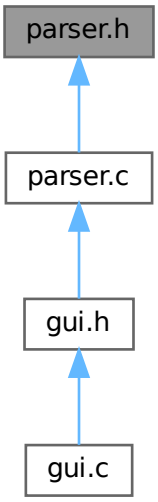
parser of the calculator app

```
#include <stdbool.h>
#include "math.c"
```

Include dependency graph for parser.h:



This graph shows which files directly or indirectly include this file:



Functions

- double parse_factorial (const char ∗equation, unsigned int lenght, unsigned int ∗error, bool ∗parsed)

  searches the string equation for factorials and evaluates them
- double parse_power (const char ∗equation, unsigned int lenght, unsigned int ∗error, bool ∗parsed)

  searches the string equation for powers and evaluates them

- double parse_root (const char ∗equation, unsigned int lenght, unsigned int ∗error, bool ∗parsed)

  searches the string equation for roots and evaluates them
- double parse_log (const char ∗equation, unsigned int lenght, unsigned int ∗error, bool ∗parsed)

  searches the string equation for logarithms and evaluates them
- double parse_mul (const char ∗equation, unsigned int lenght, unsigned int ∗error, bool ∗parsed)

  searches the string equation for multiplications and evaluates them
- double parse_div (const char ∗equation, unsigned int lenght, unsigned int ∗error, bool ∗parsed)

  searches the string equation for divisions and evaluates them
- double parse_add (const char ∗equation, unsigned int lenght, unsigned int ∗error, bool ∗parsed)

  searches the string equation for additions and evaluates them
- double parse_sub (const char ∗equation, unsigned int lenght, unsigned int ∗error, bool ∗parsed)

  searches the string equation for subtractions and evaluates them
- double parse_equation (const char ∗equation, unsigned int lenght, unsigned int depth, unsigned int ∗error)

  parses a string into a mathematical equation and returns its value
- double parse_number (const char ∗number, unsigned int lenght, unsigned int ∗error)

  parses a string into decimal number

2.10.1 Detailed Description

parser of the calculator app

Author

Daniel Kratky

This file declares function prototypes for functions that handle user input

2.10.2 Function Documentation

2.10.2.1 parse_add()

```
double parse_add (
            const char * equation,
            unsigned int lenght,
            unsigned int * error,
            bool * parsed )
```

searches the string equation for additions and evaluates them

Parameters

equation	Pointer to the string containing the equation
lenght	Lenght of the string containing the equation
error	Pointer to error code (0 if no error occurred)
parsed	Pointer bool set to true if the function parsed anythng, false otherwise

Returns

value of the equation

2.10.2.2 parse_div()

```
double parse_div (
            const char * equation,
            unsigned int lenght,
            unsigned int * error,
            bool * parsed )
```

searches the string equation for divisions and evaluates them

Parameters

equation	Pointer to the string containing the equation
lenght	Lenght of the string containing the equation
error	Pointer to error code (0 if no error occurred)
parsed	Pointer bool set to true if the function parsed anythng, false otherwise

Returns

value of the equation

2.10.2.3 parse_equation()

```
double parse_equation (
            const char * equation,
            unsigned int lenght,
            unsigned int depth,
            unsigned int * error )
```

parses a string into a mathematical equation and returns its value

Parameters

equation	Pointer to the string containing the equation
lenght	Lenght of the string containing the equation
depth	Second addend
error	Pointer to error code (0 if no error occurred)

Returns

value of the equation

2.10.2.4 parse_factorial()

```
double parse_factorial (
            const char * equation,
```

```
            unsigned int lenght,
            unsigned int * error,
            bool * parsed )
```

searches the string equation for factorials and evaluates them

Parameters

equation	Pointer to the string containing the equation
lenght	Lenght of the string containing the equation
error	Pointer to error code (0 if no error occurred)
parsed	Pointer bool set to true if the function parsed anythng, false otherwise

Returns

value of the equation

2.10.2.5 parse_log()

```
double parse_log (
            const char * equation,
            unsigned int lenght,
            unsigned int * error,
            bool * parsed )
```

searches the string equation for logarithms and evaluates them

Parameters

equation	Pointer to the string containing the equation
lenght	Lenght of the string containing the equation
error	Pointer to error code (0 if no error occurred)
parsed	Pointer bool set to true if the function parsed anythng, false otherwise

Returns

value of the equation

2.10.2.6 parse_mul()

```
double parse_mul (
            const char * equation,
            unsigned int lenght,
            unsigned int * error,
            bool * parsed )
```

searches the string equation for multiplications and evaluates them

Parameters

equation	Pointer to the string containing the equation
lenght	Lenght of the string containing the equation
error	Pointer to error code (0 if no error occurred)
parsed	Pointer bool set to true if the function parsed anythng, false otherwise

Returns

value of the equation

2.10.2.7 parse_number()

```
double parse_number (
            const char * number,
            unsigned int lenght,
            unsigned int * error )
```

parses a string into decimal number

Parameters

number	Pointer to the string containing the number
lenght	Lenght of the string containing the equation
depth	Second addend
error	Pointer to error code (0 if no error occurred)

Returns

decimal value of the number

2.10.2.8 parse_power()

```
double parse_power (
            const char * equation,
            unsigned int lenght,
            unsigned int * error,
            bool * parsed )
```

searches the string equation for powers and evaluates them

Parameters

equation	Pointer to the string containing the equation
lenght	Lenght of the string containing the equation
error	Pointer to error code (0 if no error occurred)
parsed	Pointer bool set to true if the function parsed anythng, false otherwise

Returns

value of the equation

2.10.2.9 parse_root()

```
double parse_root (
            const char * equation,
            unsigned int lenght,
            unsigned int * error,
            bool * parsed )
```

searches the string equation for roots and evaluates them

Parameters

equation	Pointer to the string containing the equation
lenght	Lenght of the string containing the equation
error	Pointer to error code (0 if no error occurred)
parsed	Pointer bool set to true if the function parsed anythng, false otherwise

Returns

value of the equation

2.10.2.10 parse_sub()

```
double parse_sub (
            const char * equation,
            unsigned int lenght,
            unsigned int * error,
            bool * parsed )
```

searches the string equation for subtractions and evaluates them

Parameters

equation	Pointer to the string containing the equation
lenght	Lenght of the string containing the equation
error	Pointer to error code (0 if no error occurred)
parsed	Pointer bool set to true if the function parsed anythng, false otherwise

Returns

value of the equation

2.11 parser.h

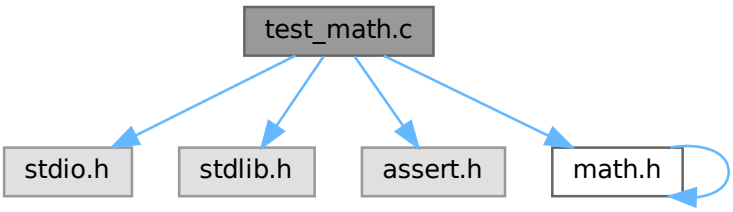
Go to the documentation of this file.

```
//======= Copyright (c) 2026, FIT VUT Brno, GNU GPL V3.0 ============//
//
// Purpose:     calculator parser
//
// $NoKeywords: $ivs_project_2 $parser.h
// $Authors:    Daniel Kratky <xkratkd00@stud.fit.vut.cz>
// $Date:       $2026-04-28
//===========================================================================//
#ifndef PARSER_H
#define PARSER_H
#include <stdbool.h>
#include "math.c"

double parse_factorial(const char *equation, unsigned int lenght, unsigned int *error, bool *parsed);

double parse_power(const char *equation, unsigned int lenght, unsigned int *error, bool *parsed);

double parse_root(const char *equation, unsigned int lenght, unsigned int *error, bool *parsed);

double parse_log(const char *equation, unsigned int lenght, unsigned int *error, bool *parsed);

double parse_mul(const char *equation, unsigned int lenght, unsigned int *error, bool *parsed);

double parse_div(const char *equation, unsigned int lenght, unsigned int *error, bool *parsed);

double parse_add(const char *equation, unsigned int lenght, unsigned int *error, bool *parsed);

double parse_sub(const char *equation, unsigned int lenght, unsigned int *error, bool *parsed);

double parse_equation(const char *equation, unsigned int lenght, unsigned int depth, unsigned int
*error);

double parse_number(const char *number, unsigned int lenght, unsigned int *error);
#endif // PARSER_H
```

2.12 test_math.c File Reference

```
#include <stdio.h>
#include <stdlib.h>
#include <assert.h>
#include <math.h>
```

Include dependency graph for test_math.c:



Functions

- double add (double a, double b)
- double sub (double a, double b)
- double mul (double a, double b)
- double divide (double a, double b)
- long long factorial (int n)
- double power (double base, int exponent)

- double n_root (double degree, double radicand)
- double absolute (double a)
- double logarithm (double x)
- int is_equal (double a, double b)
- int main ()

2.12.1 Function Documentation

2.12.1.1 absolute()

```
double absolute (
            double a )
```

2.12.1.2 add()

```
double add (
            double a,
            double b )
```

2.12.1.3 divide()

```
double divide (
            double a,
            double b )
```

2.12.1.4 factorial()

```
long long factorial (
            int n )
```

2.12.1.5 is_equal()

```
int is_equal (
            double a,
            double b )
```

2.12.1.6 logarithm()

```
double logarithm (
            double x )
```

2.12.1.7 main()

```
int main ( )
```

2.12.1.8 mul()

```
double mul (
            double a,
            double b )
```

2.12.1.9 n_root()

```
double n_root (
            double degree,
            double radicand )
```

2.12.1.10 power()

```
double power (
            double base,
            int exponent )
```

2.12.1.11 sub()

```
double sub (
            double a,
            double b )
```

Index

absolute
    test_math.c, 40
add
    math.c, 11
    math.h, 19
    test_math.c, 40

divide
    math.c, 11
    math.h, 19
    test_math.c, 40

err_msg
    gui.c, 5

factorial
    math.c, 12
    math.h, 20
    test_math.c, 40

gui.c, 3
    err_msg, 5
    main, 4
    on_backspace_clicked, 4
    on_button_clicked, 4
    on_clear_clicked, 4
    on_key_press, 5
    run_calculator, 5
gui.h, 6
    on_backspace_clicked, 7
    on_button_clicked, 7
    on_clear_clicked, 8
    on_key_press, 8
    run_calculator, 8

is_equal
    test_math.c, 40

logarithm
    math.c, 12
    math.h, 20
    test_math.c, 40

main
    gui.c, 4
    test_math.c, 40
math.c, 9
    add, 11
    divide, 11
    factorial, 12
    logarithm, 12
    mul, 12
    n_root, 13
    power, 13
    sub, 14
math.h, 17
    add, 19
    divide, 19
    factorial, 20
    logarithm, 20
    mul, 20
    n_root, 21
    power, 21
    sub, 21
mul
    math.c, 12
    math.h, 20
    test_math.c, 40

n_root
    math.c, 13
    math.h, 21
    test_math.c, 41

on_backspace_clicked
    gui.c, 4
    gui.h, 7
on_button_clicked
    gui.c, 4
    gui.h, 7
on_clear_clicked
    gui.c, 4
    gui.h, 8
on_key_press
    gui.c, 5
    gui.h, 8

parse_add
    parser.c, 24
    parser.h, 34
parse_div
    parser.c, 24
    parser.h, 35
parse_equation
    parser.c, 25
    parser.h, 35
parse_factorial
    parser.c, 25
    parser.h, 35
parse_log
    parser.c, 26

parser.h, 36
parse_mul
parser.c, 26
parser.h, 36
parse_number
parser.c, 27
parser.h, 37
parse_power
parser.c, 27
parser.h, 37
parse_root
parser.c, 27
parser.h, 38
parse_sub
parser.c, 28
parser.h, 38
parser.c, 22
parse_add, 24
parse_div, 24
parse_equation, 25
parse_factorial, 25
parse_log, 26
parse_mul, 26
parse_number, 27
parse_power, 27
parse_root, 27
parse_sub, 28
parser.h, 32
parse_add, 34
parse_div, 35
parse_equation, 35
parse_factorial, 35
parse_log, 36
parse_mul, 36
parse_number, 37
parse_power, 37
parse_root, 38
parse_sub, 38
power
math.c, 13
math.h, 21
test_math.c, 41

run_calculator
gui.c, 5
gui.h, 8

sub
math.c, 14
math.h, 21
test_math.c, 41

test_math.c, 39
absolute, 40
add, 40
divide, 40
factorial, 40
is_equal, 40
logarithm, 40
main, 40
mul, 40
n_root, 41
power, 41
sub, 41